



Artifact Lifecycle Control Plane

Architecture, Hardening & Development Direction

CONTROL-PLANE THESIS

ATLAS is evolving from a compact local pipeline kernel into a governed artifact-processing substrate that preserves identity, provenance, safety, evidence, and promotion decisions from raw source material to trusted downstream use.

REPOSITORY REFERENCE

Runndownn/ATLAS
main @ efa547cafc64...

CURRENT MATURITY

Pre-Alpha v0.1.0
Python 3.13+ / MIT

SEMANTIC SPINE

Recon -> Fingerprint -> Structure
Extract -> Analyze -> Review

The development direction in one view

ATLAS is evolving from a compact local artifact-processing kernel into a governed control plane that can take raw or untrusted material and preserve a reproducible chain from source, to exact bytes, to structure, to analysis, to evidence, to an explicit downstream trust decision.

The differentiator is not that ATLAS can run six Python stages. The differentiator is that it can make artifact identity, provenance, safety, evidence, and promotion decisions part of the execution model itself.

01

Keep the lifecycle stable

Reconnaissance, Fingerprinting, Structural Discovery, Controlled Extraction, Deep Understanding, and Review/Promotion remain the semantic spine even as the runtime grows.

02

Make semantics durable

The immediate work turns intake, content identity, progress, events, error policy, control requests, and checkpoints into persistent runtime truth instead of partial or process-local behavior.

03

Govern materialization and trust

Unsafe containers are inspected before extraction; extracted content moves to quarantined workspaces; analysis findings remain distinct from approved knowledge or publications.

04

Scale only after integrity

Distributed workers, PostgreSQL, content-addressable/object storage, APIs, MCP, and operator surfaces follow after local semantics and recovery behavior are coherent.

6

stable lifecycle phases

3

P0 workstreams: semantics, safety, durable execution

4

P1 workstreams: reliability, evidence, analyzers, integration

2

P2 workstreams: scale and operator experience

END-STATE THESIS

ATLAS should become an artifact lifecycle control plane, not another generic workflow or agent framework. Agents, LLMs, parsers, security tools, and domain analyzers can operate inside governed phases; ATLAS remains authoritative for lifecycle state, provenance, safety, evidence lineage, policy, and promotion.

Source basis: the supplied ATLAS development-direction material for **Runndownn/ATLAS**, frozen at **main @ efa547cafc64b58d60d19dcc1e5ba32c6b046bc5**. The report preserves the source's distinction between current behavior and intended architecture.

What ATLAS is - and what it is not

What it is

ATLAS is a **pre-alpha artifact lifecycle engine**. Its job is to move potentially untrusted files, archives, repositories, documents, security evidence, and related inputs through a controlled series of state transitions.

It originated from processing concepts in Yggdrasil and Geezer Mekanix, but deliberately strips away the larger platform concerns that surrounded those concepts - authentication, RBAC, multi-tenancy, secrets management, cognitive operations, TLS, and other infrastructure.

What remains is the reusable processing kernel: orchestration, state, events, discovery, hashing, archive safety, analysis extension points, and review/promotion concepts.

What it is not

Not a general DAG scheduler

The runtime is asynchronous, but the artifact lifecycle is deliberately ordered and sequential.

Not a task queue

Current execution is local and process-oriented; durable worker dispatch is an intended later stage.

Not an agent framework

AI and agent systems are expected to run inside bounded analysis/review roles rather than control provenance or trust transitions.

Not yet a knowledge fabric

Evidence and promotion concepts exist, but persistent findings, review decisions, destination adapters, and full lineage remain development work.

ARCHITECTURAL ROLE

ATLAS is best positioned as the **governed ingestion and artifact-processing layer underneath** security systems, forensic workflows, repository intelligence, knowledge pipelines, corpus curation, and AI-assisted analysis.

Why the boundary matters

Generic orchestration answers "which task runs next?" ATLAS is intended to answer a more evidence-sensitive question: **what happened to this exact content, under which controls, and why was a result ever trusted?**

Identity-aware

Provenance-aware

Safety-first

Evidence-aware

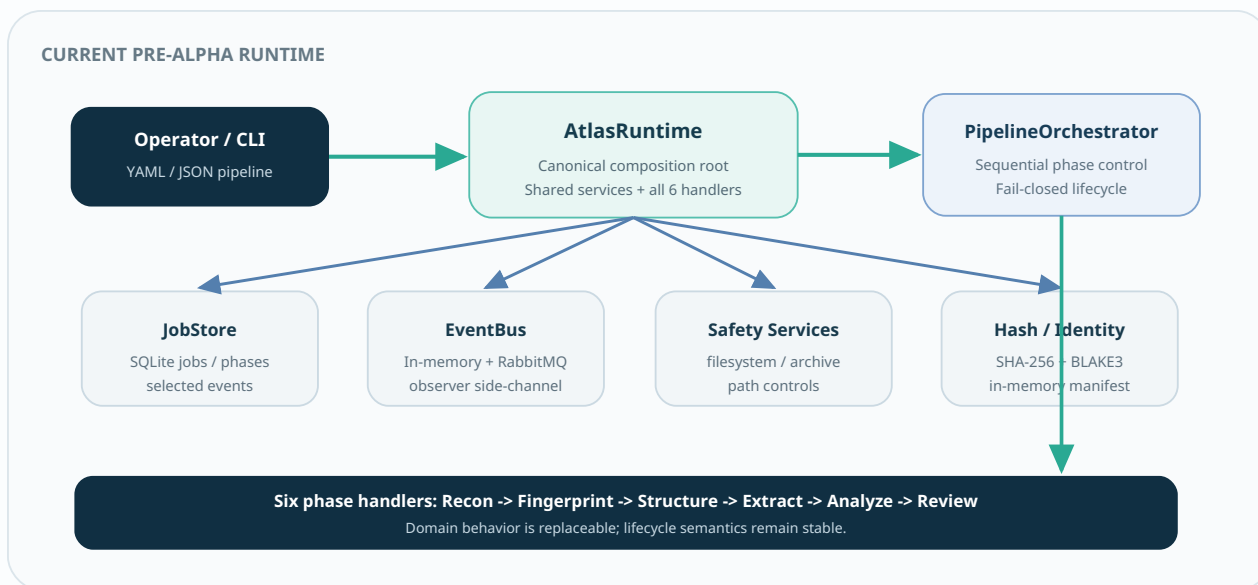
Review-gated

Extensible analysis

A compact, readable execution kernel

The current design is small by intent. **AtlasRuntime** centralizes construction; **PipelineOrchestrator** advances the lifecycle; SQLite records jobs and phases; safety and hashing services support phase work; the event bus decouples observers from execution.

Figure 1. Current pre-alpha runtime structure



Canonical composition

AtlasRuntime prevents CLI/tests/future APIs from accidentally constructing different handler graphs. This addresses an earlier fail-open wiring problem.

Deterministic control flow

The orchestrator walks one configured phase list in order. That simplicity is a feature for evidence-sensitive processing because the sequence remains reconstructable.

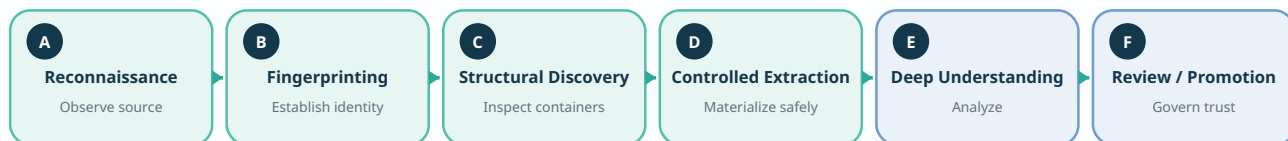
CURRENT DURABILITY

Jobs and phase records are persisted in SQLite. Orchestrator lifecycle events also have durable representation. Detailed phase-emitted observations remain more transient and are part of the hardening direction.

The six-phase semantic spine

The A-F lifecycle carries more architectural meaning than the current implementation depth. It defines how an artifact progresses from unknown input toward governed downstream use.

Figure 2. ATLAS artifact lifecycle



Stable semantic lifecycle: the implementation may evolve, but the A-F trust progression remains the architectural spine.

A

Reconnaissance

Observe source material, inventory files, record risk indicators, and establish the intake boundary.

B

Fingerprinting

Compute cryptographic identity and distinguish exact content from the path or source occurrence where it appeared.

C

Structural Discovery

Inspect archive/container organization before materializing contents.

D

Controlled Extraction

Materialize accepted content under path, archive, resource, and workspace controls.

E

Deep Understanding

Run domain analyzers and convert raw artifacts into structured findings.

F

Review / Promotion

Separate analyzer output from the policy or human decision that allows it to enter a trusted destination.

Design principle: arbitrary DAG behavior can be added around or within lifecycle stages later. The lifecycle itself should remain stable so provenance and trust transitions stay intelligible.

A-C: establish observation, identity, and structure

A

Reconnaissance

Today: filesystem walk, file/byte counts, archive detection, executable indicators, sensitive-looking paths, symlink awareness.

Target: emit an immutable *IntakeGeneration* so every later phase works from a stable observation instead of a source tree that may change during processing.

B

Fingerprinting

Today: streaming SHA-256 and optional BLAKE3 hashing, with an in-memory manifest.

Target: persist *ContentIdentity* independently from *ArtifactOccurrence*, enabling cross-run duplicate recognition without losing provenance.

C

Structural Discovery

Today: inspect archive members, suspicious paths, and nested-archive presence without extraction.

Target: recursively understand nested containers and packaging relationships under strict depth, member, and byte budgets.

The key semantic correction: occurrence is not identity

ATLAS must treat a filesystem path as an **occurrence** and a digest as the **identity of the bytes**. The same bytes can appear in a repository, a ZIP, a backup, an email attachment, and a forensic image. Those locations should remain separately traceable while converging on one content identity.

WHY IT MATTERS

Persistent occurrence-to-content relationships enable true deduplication, repeat-analysis avoidance, lineage queries, migration validation, and evidence defensibility without collapsing distinct provenance into one record.

Immutable intake as a trust anchor

If fingerprinting re-discovers a live filesystem instead of consuming the exact inventory observed during Reconnaissance, the source can change between phases. The long-term design therefore treats Reconnaissance as a snapshot authority: downstream phases operate on the recorded occurrence set, not a fresh traversal.

D-F: controlled materialization, analysis, and trust

D

Controlled Extraction

Today: reassess archives, block unsafe results, enforce destination containment, and use safer TAR data filtering.

Target: isolated per-job quarantine workspaces governed by one canonical path/archive policy.

E

Deep Understanding

Today: basic metadata plus an analyzer plugin protocol.

Target: analyzer registry, normalized finding schema, declared capabilities/resources, plugin isolation, and rich domain-specific analysis.

F

Review / Promotion

Today: evidence-oriented records and candidate promotion concepts.

Target: durable review decisions, policy gates, destination adapters, and traceable publication history.

Processing something is not the same as deciding that a derived assertion deserves to become trusted knowledge.

Evidence semantics are strategically important

The review model already distinguishes direct observations, parsed facts, inferred relationships, and classification hypotheses. That is a stronger foundation than a generic "result" object because downstream systems can reason about **how a claim was produced**, not just its text.

ANALYZER RESPONSIBILITY

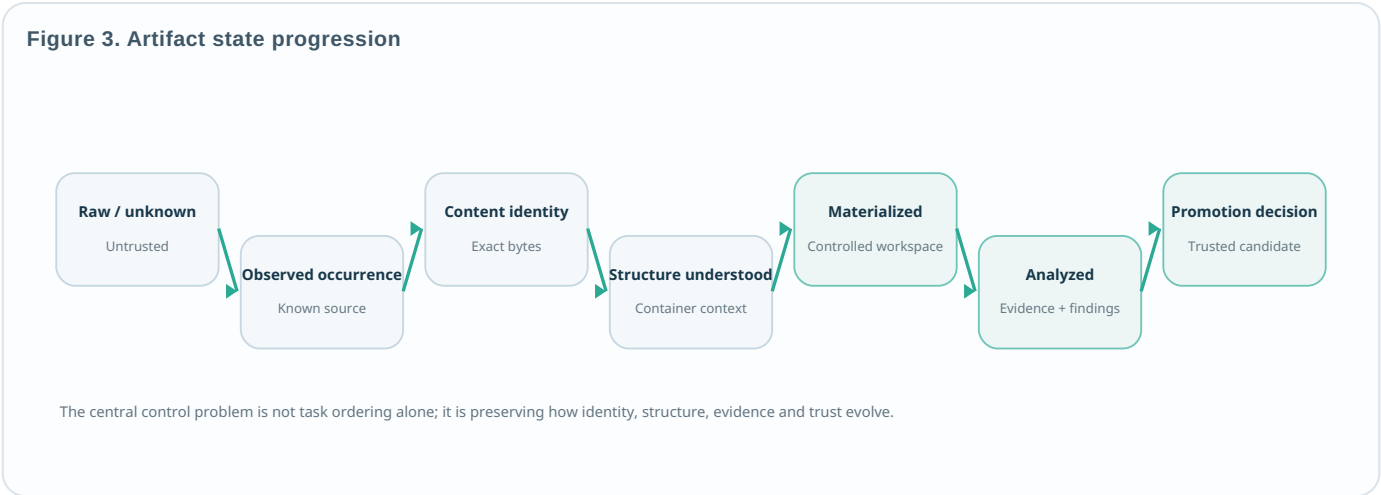
Produce normalized findings, tool/model attribution, relevant source references, confidence where appropriate, and enough context for review.

ATLAS RESPONSIBILITY

Preserve provenance, maintain lifecycle state, enforce policy boundaries, record review outcomes, and control publication into trusted destinations.

The system is managing trust state, not only tasks

The core conceptual shift is from a task graph to an artifact state model. Each phase should leave behind durable evidence of what changed in the system's understanding or authority over the artifact.



State transition	What becomes known or controlled	Primary record
Unknown -> observed	Where the material was seen, at what intake generation, with which source metadata	ArtifactOccurrence
Observed -> identified	Exact byte identity independent of location	ContentIdentity
Identified -> structured	Container members, nesting, suspicious paths, packaging relationships	StructuralReport
Structured -> materialized	Which content was created on disk, where, and under which safety policy	ExtractionRecord
Materialized -> analyzed	Normalized tool/model findings and relationships	AnalysisFinding
Analyzed -> governed	What evidence supports a decision and whether promotion was allowed	Evidence + PromotionDecision

This state model is what allows ATLAS to answer provenance questions later: origin, exact bytes, extraction lineage, analyzer attribution, confidence, review decision, and destination publication.

What is already strong

Clear composition root

AtlasRuntime gives the CLI, tests, and future API layers one canonical way to construct the engine and all six handlers.

Fail-closed phase dispatch

Configured phases without a handler are treated as errors instead of being silently marked successful.

Readable orchestration

The execution path is compact and direct. For evidence-sensitive software, that readability supports assurance and debugging.

Durable job and phase state

SQLite stores job and phase lifecycle records, creating a useful baseline for inspection and future recovery semantics.

Safety is a separate domain

Filesystem discovery, archive safety, and path checks are not buried in ad hoc phase code, which supports defense-in-depth evolution.

Streaming identity

SHA-256 and optional BLAKE3 are computed in chunks, giving a scalable content-identity primitive even though persistent CAS is not implemented yet.

Analysis is extensible

The analyzer plugin protocol creates a natural seam for security, repository, document, forensic, and AI analysis engines.

Trust is modeled explicitly

Evidence and promotion concepts create the right boundary between producing a result and treating it as authoritative.

Hardening has already changed the trajectory

The source describes an earlier architecture review that exposed fail-open handler wiring, structural-discovery/extraction conflation, TAR safety issues, event persistence gaps, duplicate-key behavior, serialization problems, and configuration issues. Subsequent code work addressed several of those findings. That assessment-to-hardening feedback loop should remain part of the project culture.

STRATEGIC ADVANTAGE

ATLAS is still small enough that semantics can be corrected before distributed infrastructure and broad integrations make those mistakes expensive to unwind.

Where the runtime and the architecture still diverge

Area	Current behavior	Required direction
Phase configuration	Stored in metadata; handlers do not consistently consume it	Typed, validated, injected, enforced policy
Error policy	Overlapping continue/abort concepts	One authoritative runtime contract
Content store	In-memory hash manifest	Persistent content + occurrence model
Deduplication	Same bytes produce same hash; repeat work is not durably avoided	Cross-run identity reuse with provenance
Progress	Phase object changes are not continuously persisted as live job progress	Durable phase + aggregate progress
Pause / resume / cancel	Depends on process-local runtime state	Persistent control requests + worker cooperation
Detailed events	Phase-emitted observations may remain transient	Durable operational/evidentiary event stream
RabbitMQ	Partial publishing/fallback capability	Defined consumption, backpressure, delivery semantics
Nested archives	Detected and counted	Recursive bounded inspection
Analysis	Plugin seam + basic metadata	Registry + normalized findings + isolation
Promotion	Conceptual evidence/candidates	Persistent reviews + destination publication
Recovery	State recorded; no complete checkpoint/retry model	Idempotency, timeout, checkpoint, crash recovery

INTERPRETATION

These are not reasons to abandon the architecture. They are the exact seams where the pre-alpha implementation must become semantically honest before ATLAS expands into a multi-process or distributed service.

Build integrity before scale

The development direction contains nine major workstreams. Their sequencing is deliberate: first make the local lifecycle truthful and durable; then operationalize evidence and integrations; only then distribute the system.



Figure 4. Development workstreams by source-defined priority. Counts are from the supplied execution rewrite, not a performance score.

P0

Semantic integrity. Immutable discovery manifest; persistent content/occurrence model; true dedup; configuration enforcement; live progress; one error policy.

Safety and evidence. Quarantine workspaces; canonical path safety; recursive archive budgets; durable detailed events.

Durable execution. Daemon/workers; persisted control requests; cooperative pause/cancel; checkpoints.

P1

Reliability. retry/backoff/timeout/idempotency/crash recovery/migrations.

Knowledge model. persistent findings, evidence, relationships, reviews and promotions.

Analyzer ecosystem. registry, normalized schema, capabilities/resources, isolation.

Integration. RabbitMQ semantics, REST, MCP, webhooks and event adapters.

P2

Distributed scale. PostgreSQL projections, object/CAS storage, worker pools, quotas.

Operator surface. Job Status Surface, review queue, artifact/evidence graph, provenance viewer.

Establish semantic integrity

The first P0 objective is simple to state but foundational: every concept exposed by ATLAS configuration and APIs must correspond to one clear runtime meaning.

Immutable discovery manifest

Reconnaissance should produce a durable intake generation containing the exact occurrence set that later phases consume. Downstream phases should not silently re-scan a changing source.

Persistent occurrence + identity

Store filesystem/source occurrences separately from cryptographic content identities. Many occurrences may reference one content identity; neither concept replaces the other.

Real duplicate avoidance

Use persistent identity to prevent unnecessary repeat processing while retaining every occurrence and its source provenance.

Runtime-enforced phase configuration

Values such as discovery depth, hash algorithms/chunk size, extraction byte ceilings, and archive-member limits must be typed, validated, passed to handlers, and enforced.

One failure-policy contract

Remove ambiguity between "continue after phase error" and "abort on error." A pipeline definition should predict execution behavior exactly.

Live progress as durable state

Persist intermediate phase progress during execution and derive meaningful job-level progress so a separate status process sees real activity.

DEFINITION OF DONE

Intake state, identity, deduplication, configuration, progress, and failure policy all have one durable meaning that is shared by the CLI, embedded API, tests, workers, and future remote interfaces.

Why this must precede scale

Distributing workers or adding APIs before semantic integrity would replicate ambiguity across components. P0-1 makes the local engine authoritative enough to become a service safely.

Strengthen safety and evidence preservation

Per-job quarantine workspaces

Controlled Extraction should materialize accepted content into isolated job-specific workspaces, not adjacent to the source archive. The workspace becomes a managed side-effect boundary.

One canonical path-safety policy

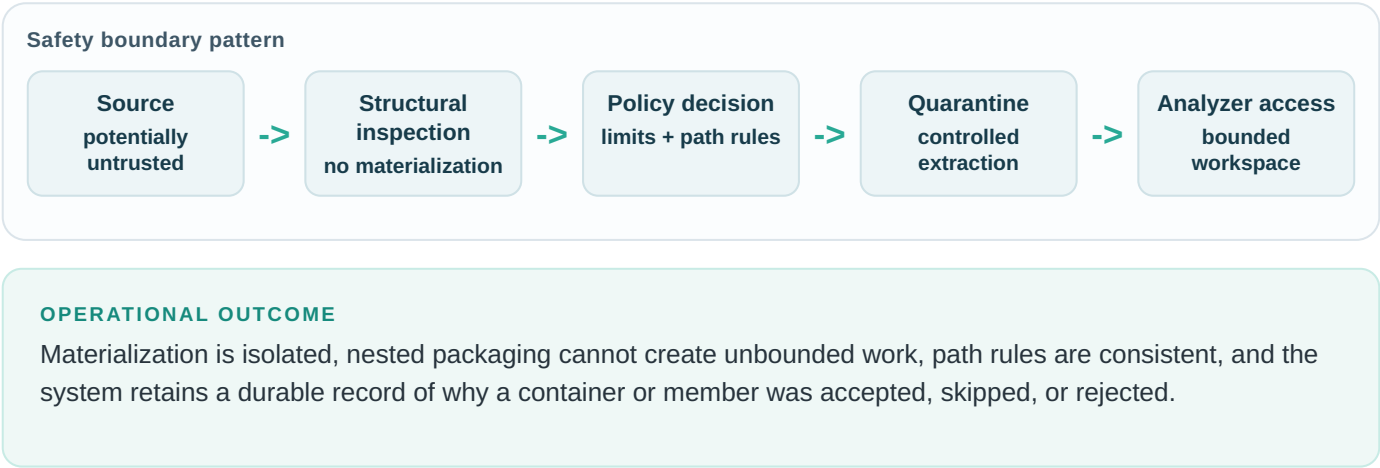
Discovery, structural inspection, extraction, analyzer file access, and future publication adapters should use the same containment and risky-path rules.

Recursive container budgets

Nested archives need bounded recursive structural inspection. Limits should cover nesting depth, total expanded bytes, member counts, and other resource ceilings.

Durable detailed events

Phase-level observations such as discovery progress, analyzer activity, extraction decisions, and errors should join the durable event history instead of existing only on a transient bus.



Make execution durable

Today, durable inspection and process-local execution control are different things. The P0 execution work closes that gap.

1

Long-running runtime

Introduce a daemon or worker process that owns active jobs instead of rebuilding execution context for every CLI invocation.

2

Persisted control requests

Pause, resume, and cancel become durable commands stored or queued for the active worker to consume.

3

Cooperative phase control

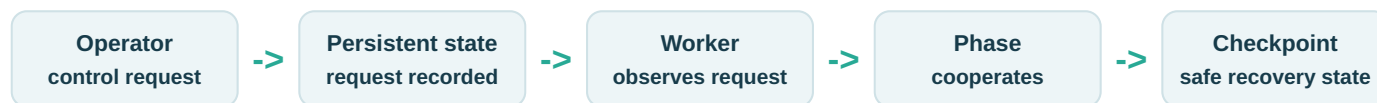
Long-running handlers periodically observe cancellation/pause state and stop at defined safe boundaries.

4

Checkpoints

Record recovery points so resumption is not equivalent to blindly rerunning a phase whose side effects may already have occurred.

Durable control loop



DEFINITION OF DONE

A second process can inspect a durable job, submit a control request, and have the active runtime honor it in a defined way. Process loss no longer erases the control context needed to resume safely.

Define retry, timeout, idempotency, and recovery

Once ATLAS owns durable jobs, it must define what a failure means for each phase - especially when a phase has filesystem or external side effects.

Control	Purpose	Design question ATLAS must answer
Retry policy	Recover transient failures	Which failures are retryable, and how many attempts are allowed?
Backoff	Avoid hot failure loops	How does retry cadence adapt to repeated failure or external dependency pressure?
Timeouts	Bound long-running operations	What is the phase-level deadline and what cleanup occurs on expiry?
Idempotency	Prevent duplicate side effects	Can the phase be replayed safely from its last durable checkpoint?
Crash recovery	Resume after process loss	Which persistent state proves what work completed before the crash?
Schema migrations	Evolve durable state safely	How are version changes applied and rolled back without corrupting jobs?

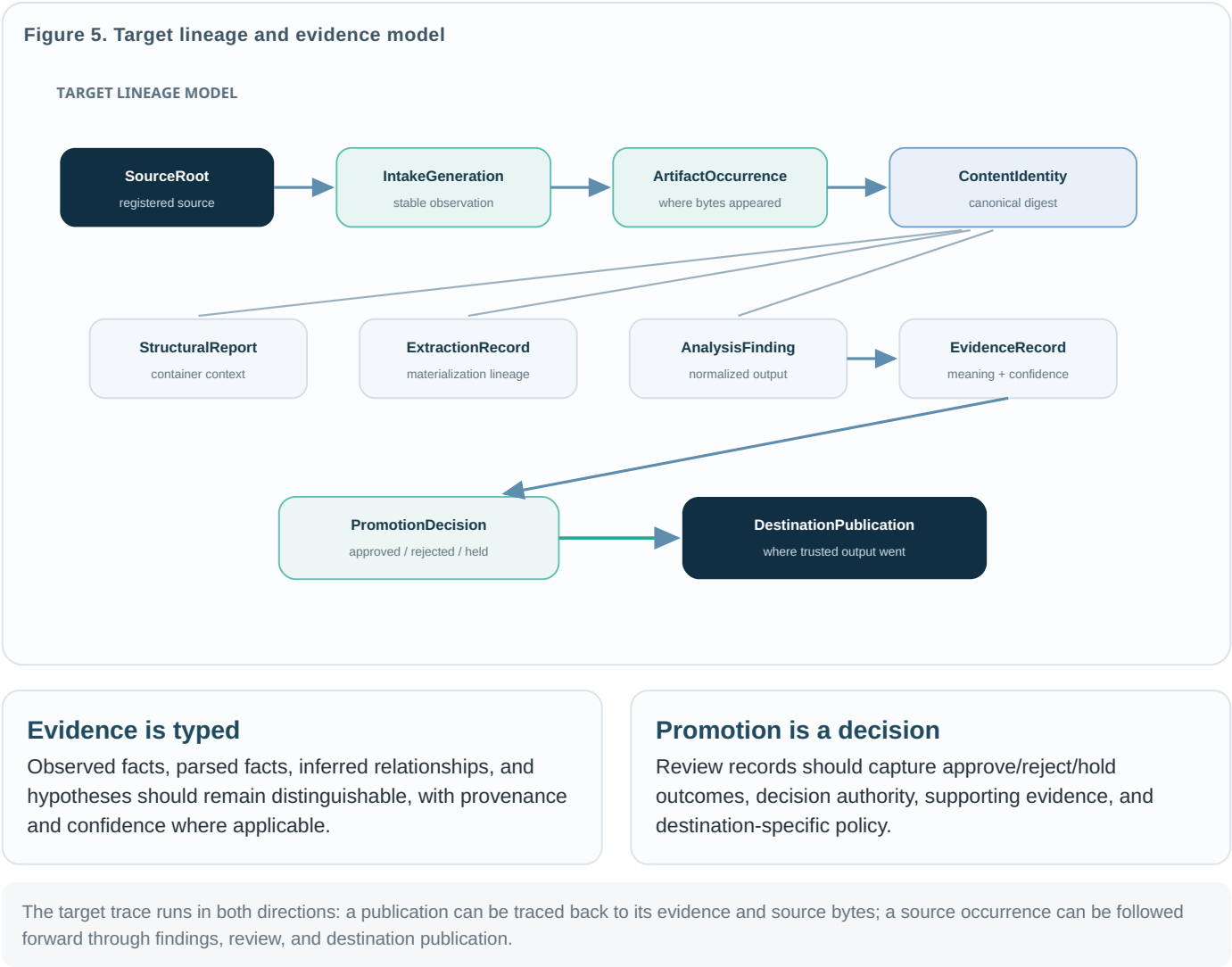
Automatic retry is only safe when ATLAS can prove whether repeating the phase will duplicate a side effect or corrupt lineage.

COMPLETION CONDITION

Every built-in phase has explicit repeat-execution semantics, recovery state survives process interruption, and database evolution is versioned rather than implicit.

Make findings, evidence, review, and publication first-class

The mature control plane needs a persistent lineage model that connects source material to accepted downstream output. Jobs and phase records alone are not enough.



Turn Deep Understanding into a governed plugin platform

Phase E is the extension point through which ATLAS becomes useful across security, repositories, documents, forensics, and AI-assisted workflows. The plugin seam therefore needs a contract stronger than "return any dictionary."

Analyzer registry

Discover and register analyzers by capability, artifact type, domain, version, and supported inputs.

Normalized finding schema

Give all analyzers a shared envelope for severity/ category, observation, source reference, confidence, relationships, tool version, timestamps, and structured data.

Capability declarations

Plugins should declare filesystem access, network needs, CPU/memory profile, external tools, and whether they can execute active content.

Resource controls

The runtime can enforce quotas and decide which worker or sandbox is appropriate for each analyzer.

Plugin isolation

A parser or analyzer should not automatically inherit unrestricted authority over the ATLAS runtime or host filesystem.

Versioned attribution

Findings should record the analyzer/tool/model version so later review can reproduce or reinterpret results.

ARCHITECTURAL RULE

Domain intelligence belongs inside controlled plugins. Identity, safety, provenance, lifecycle state, and promotion authority remain core ATLAS responsibilities.

Examples of Phase E analyzers

YARA

Capa

FLOSS

file-format parsers

repository analyzers

document extractors

log parsers

PCAP analyzers

LLM analysis

External systems should use the lifecycle, not bypass it

Complete event transport

RabbitMQ should move from partial publishing/fallback behavior toward defined connection lifecycle, consumers/subscriptions, delivery semantics, and backpressure.

REST service

A service interface can submit pipeline jobs, inspect status, retrieve evidence, and issue durable control requests.

MCP / agent-facing interface

Agents can request governed artifact processing through a constrained tool surface rather than manipulating local files or internal stores directly.

Webhooks and event adapters

External systems can observe terminal states, review requirements, publication events, and failures without being coupled to the core orchestrator.



KEY REQUIREMENT

Every external interface must preserve the same lifecycle and provenance rules as local execution. API convenience must not create a second, less-governed processing path.

Scale and operator experience come after semantic stability

Distributed scale

PostgreSQL projection

Move high-concurrency durable state beyond a single SQLite connection while preserving the same job/phase/evidence semantics.

Object or content-addressable storage

Persist materialized and canonical content independently of worker-local disks.

Worker pools + quotas

Dispatch analyzers by capability and bound resource usage per job, phase, source, or tenant context.

Operator surface

Job Status Surface

Expose durable lifecycle state, checkpoints, progress, current phase, failures, and control requests.

Review queue

Present pending promotion decisions with evidence, provenance, and destination policy in context.

Artifact/evidence graph

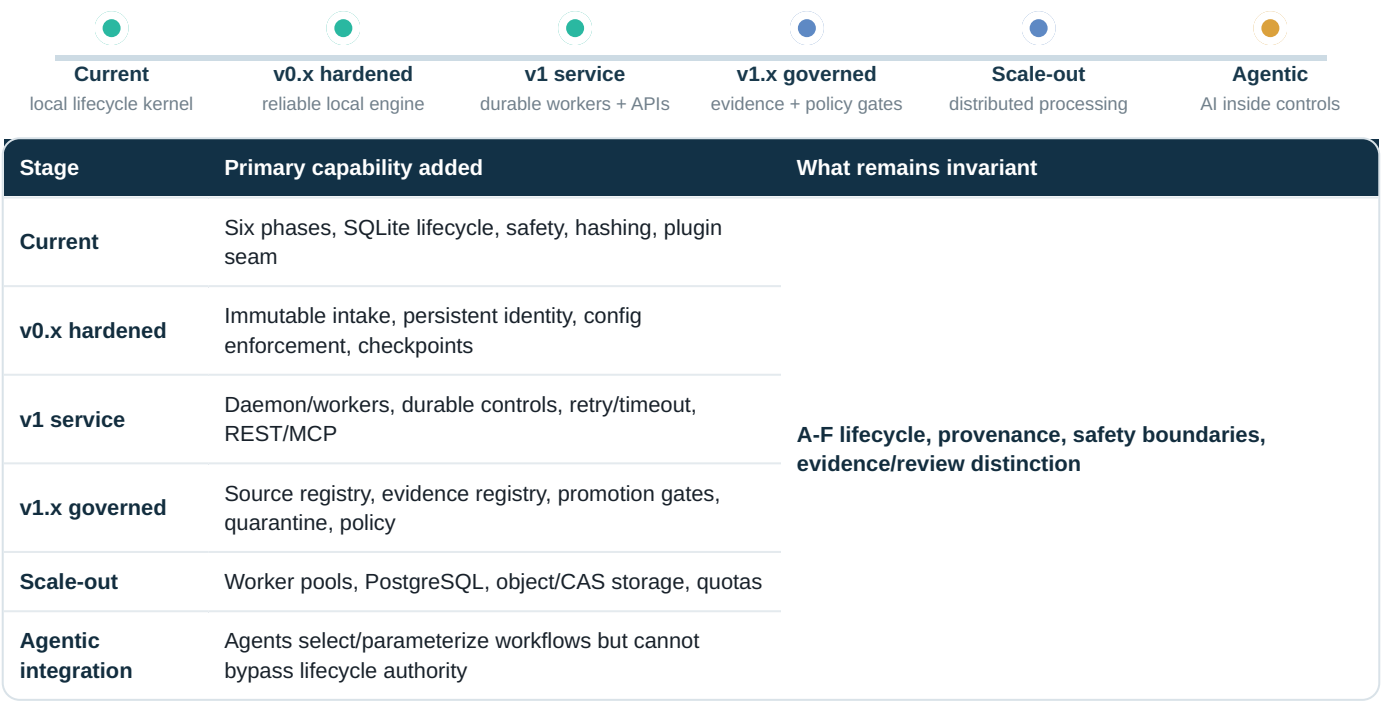
Let operators navigate occurrence -> content -> extraction
-> finding -> evidence -> decision -> publication lineage.

SEQUENCING RULE

Scale changes infrastructure, not semantics. The local engine must define identity, control, recovery, and trust correctly before those same concepts are distributed across workers and stores.

A staged path from kernel to governed control plane

The source direction describes a progressive expansion rather than a replacement architecture. Each stage adds operational capability while preserving the six-phase semantic spine.

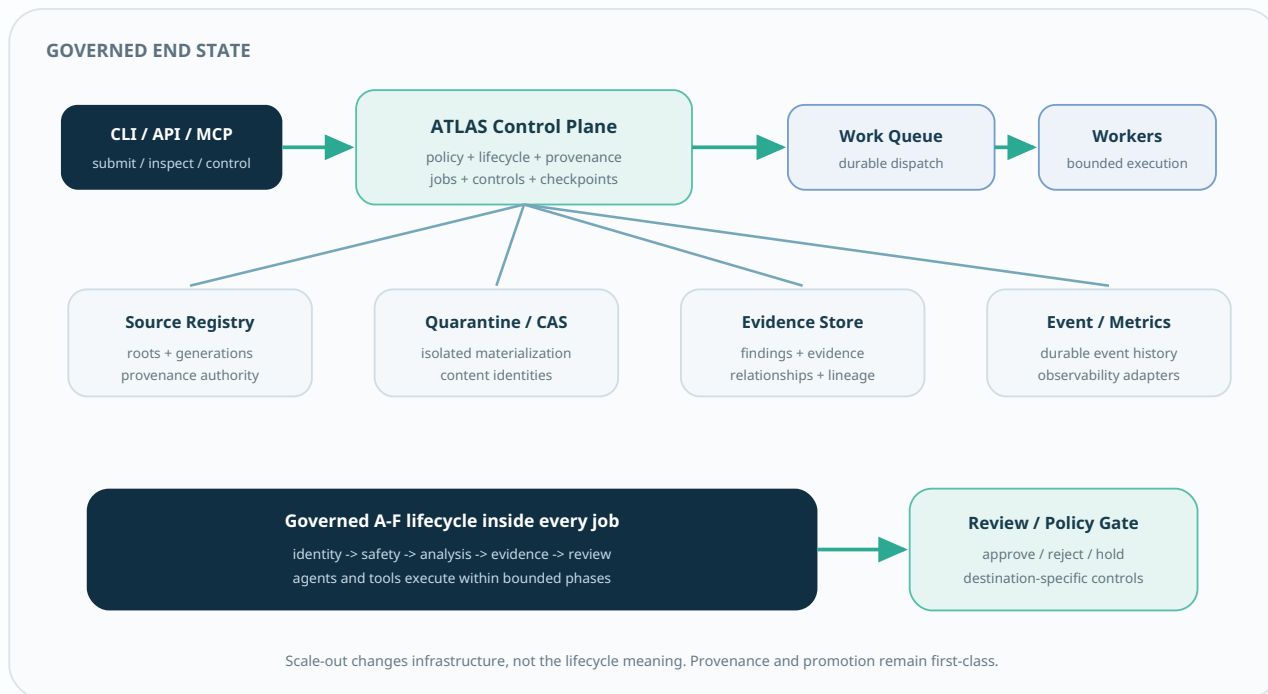


The end state is not "ATLAS becomes more autonomous." It is "ATLAS becomes more durable, inspectable, and governable while allowing more specialized systems to operate inside it."

The Artifact Lifecycle Control Plane

At maturity, ATLAS becomes the common execution and provenance substrate beneath local tools, security analyzers, knowledge systems, and AI-assisted workflows.

Figure 6. Governed end-state architecture



Control plane authority

Pipeline registration, policy, lifecycle state, durable controls, checkpoints, provenance and review decisions are centralized as authoritative records.

Worker specialization

Execution can scale out by capability while workers remain bounded by job configuration, quarantine, resource policy, and analyzer declarations.

AI belongs inside the lifecycle - not above it

The model should be able to analyze, classify, correlate, and propose. It should not become the authority over source provenance, filesystem safety, durable control, or final promotion.

Appropriate AI roles

- Classify artifact type or domain.
- Extract structured facts from documents or technical notes.
- Propose relationships between findings.
- Summarize analyzer output into evidence candidates.
- Recommend routing to specialized analyzers.
- Propose a promotion decision with supporting rationale.

ATLAS retains authority

- Register and freeze the source occurrence set.
- Establish exact content identity.
- Enforce archive and path safety.
- Control workspace materialization.
- Persist lifecycle and event history.
- Apply review/policy gates and record publication.



DESIGN OBJECTIVE

Increase analytical capability without weakening the deterministic chain of custody. AI enriches the evidence lane; ATLAS governs the evidence lifecycle.

Controls that make the architecture defensible

Control	Current / intended behavior	Why it matters
Fail-closed dispatch	Missing configured handlers fail the job	Unavailable functionality cannot masquerade as success
Pre-extraction inspection	Structure and resource risk are assessed before materialization	Untrusted containers are reasoned about before they create files
Quarantine boundary	Target: per-job isolated extraction workspaces	Controls the filesystem side effects of untrusted input
Resource budgets	Target: recursive depth, bytes, member counts, analyzer quotas	Prevents archive recursion and analysis from becoming resource exhaustion
Cryptographic identity	SHA-256, optional BLAKE3; target persistent CAS model	Anchors provenance to exact bytes, not filenames
Durable events	Lifecycle events persisted; detailed phase events targeted for persistence	Supports incident reconstruction, audit, and operational observability
Typed evidence	Observation / parsed / inferred / hypothesis distinctions	Prevents all analyzer output from being treated as equivalent truth
Promotion gate	Target durable review decisions and destination policy	Separates analysis capability from authority to publish
Recovery policy	Target checkpoint, idempotency, timeout, retry semantics	Makes automation safe under process failure and replay

GOVERNANCE PRINCIPLE

Evidence, operator decisions, access authority, and filesystem mutations should remain distinct concepts. The more ATLAS scales, the more important this separation becomes.

ADVANTAGES

Why ATLAS can be strategically useful

Versus one-off scripts

Domain tools stop rebuilding the same intake, hashing, archive, state, event, provenance, and review scaffolding.

Versus generic workflow schedulers

ATLAS can preserve artifact semantics: exact content, occurrence, structure, materialization lineage, findings, evidence, and promotion decisions.

Versus agent frameworks

Deterministic policy and lifecycle state remain outside model discretion. AI becomes a bounded capability, not the execution authority.

Versus direct ingestion into knowledge stores

ATLAS creates a governed buffer between raw/untrusted material and trusted downstream data, with explicit evidence and promotion controls.

Operational advantages once hardened

- **Repeatability:** jobs can be reconstructed from immutable intake and versioned analyzer evidence.
- **Deduplication without provenance loss:** identical bytes converge on one identity while all occurrences remain traceable.
- **Safer automation:** structural inspection and resource budgets precede extraction and analysis.
- **Cross-domain reuse:** security, repository, document, forensic, and AI analyzers share one lifecycle.
- **Decision traceability:** approved knowledge can be traced back through evidence, findings, and exact source bytes.
- **Incremental scale:** the same semantics can move from local SQLite execution to workers and service interfaces later.

MOST DEFENSIBLE POSITIONING

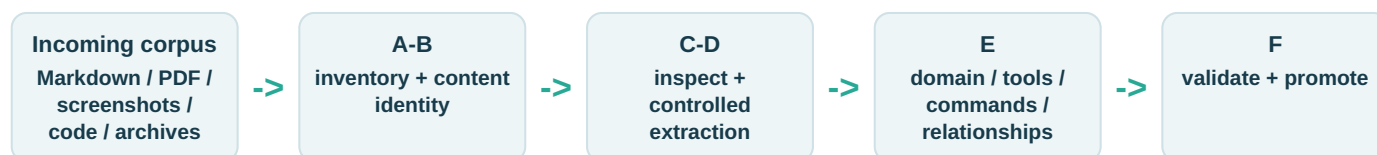
ATLAS is infrastructure for governed artifact progression. It is valuable when raw input is untrusted, repeated, nested, evidentiary, AI-analyzed, or destined for a trusted system.

Where the lifecycle model creates leverage

Use case	How ATLAS contributes
Incident-response evidence intake	Fingerprint case files, safely unpack collections, preserve provenance, dispatch forensic plugins, and gate evidence promotion.
Malware / static analysis	Quarantine samples, hash/deduplicate, inspect containers, and run YARA/Capa/FLOSS-style analyzers without executing specimens.
Network forensics	Treat PCAPs, Zeek/Suricata exports, logs, and supporting artifacts as one provenance-linked investigation.
CVE research packages	Normalize advisories, PoCs, patches, and test evidence into one reviewed vulnerability record.
Security knowledge ingestion	Deduplicate walkthroughs/notes, classify domains, extract structured techniques, and promote validated knowledge.
RAG corpus production	Separate raw-source intake from parsing, chunking, embedding, and trusted-index promotion.
Training/evaluation corpus curation	Preserve provenance, detect duplicates or contamination, and require approval before dataset admission.
Software supply chain	Hash releases, inspect packages/manifests/SBOMs/signatures, and gate artifacts before repository use.
Repository intelligence	Snapshot files, establish identities, run language/security/license analyzers, and retain findings against immutable content.
Forensics case processing	Register disk, memory, and archive artifacts and preserve lineage from tool outputs to original evidence.
Migration / backup validation	Compare source and destination content identities and qualify backups before production restoration.
Firmware / package inspection	Recursively inspect embedded containers under bounded extraction rules.

A direct fit: deterministic ingestion beneath RUDI-K / BinReaper

The supplied direction identifies security knowledge ingestion as a particularly strong fit because the corpus already contains repeatable structures such as tools and usage, techniques and methodologies, processes and steps, lessons, challenges and solutions, and potential applications.



What ATLAS would own

- Source registration and intake generation.
- Exact content identity and duplicate recognition.
- Archive and path safety.
- Controlled materialization.
- Analyzer invocation and result attribution.
- Evidence lineage and review state.

What downstream knowledge systems would own

- Canonical knowledge schema and taxonomy.
- Retrieval indexes and knowledge navigation.
- Domain-specific publication rules.
- Long-term knowledge presentation and reuse.
- Consumer-facing search and agent retrieval.

RESULT

ATLAS becomes the deterministic ingestion and provenance layer underneath the knowledge fabric rather than being limited to CTF processing.

The recommended development order

Order	Priority	Workstream	Outcome
1	P0	Semantic integrity	Immutable intake, persistent identity, real dedup, config enforcement, live progress, one error policy.
2	P0	Safety + evidence preservation	Quarantine, canonical path rules, recursive archive budgets, durable phase events.
3	P0	Durable execution	Daemon/workers, persisted controls, cooperative cancellation, checkpoints.
4	P1	Reliability controls	Retry/backoff/timeout/idempotency/crash recovery/migrations.
5	P1	Persistent evidence model	Findings, evidence, relationships, reviews, promotions, publication lineage.
6	P1	Analyzer ecosystem	Registry, normalized schema, capabilities/resources, isolation.
7	P1	External integration	RabbitMQ completion, REST, MCP, webhooks, event adapters.
8	P2	Distributed scale	PostgreSQL, object/CAS storage, worker pools, quotas.
9	P2	Operator surface	Job Status Surface, review queue, evidence/provenance navigation.

CRITICAL SEQUENCING CONSTRAINT

Do not prioritize a large web UI, multi-agent orchestration, or distributed workers before the P0 semantics are finished. Those features would scale current ambiguity instead of resolving it.

Engineering decision rule

When a proposed feature competes with a lifecycle-integrity requirement, prefer the requirement that improves provenance, recovery, policy consistency, or evidence quality. ATLAS's strategic differentiation depends on those properties remaining stronger than its orchestration novelty.

What completion should make possible

The end state can be judged by the questions ATLAS can answer reliably - without reconstructing intent from logs, filenames, or model output.

Where did this artifact originate?

Resolve SourceRoot, IntakeGeneration, and every observed occurrence.

Have these exact bytes been seen before?

Resolve a persistent ContentIdentity and all associated occurrences.

What container produced this derived file?

Follow StructuralReport and ExtractionRecord lineage.

Which analyzer made this claim?

Identify analyzer/tool/model version, input identity, and normalized finding.

Was the claim observed or inferred?

Inspect evidence type, source, confidence, and related evidence.

Who or what approved it?

Resolve the PromotionDecision, governing policy, and review authority.

Where was it published?

Resolve every DestinationPublication linked to the approval.

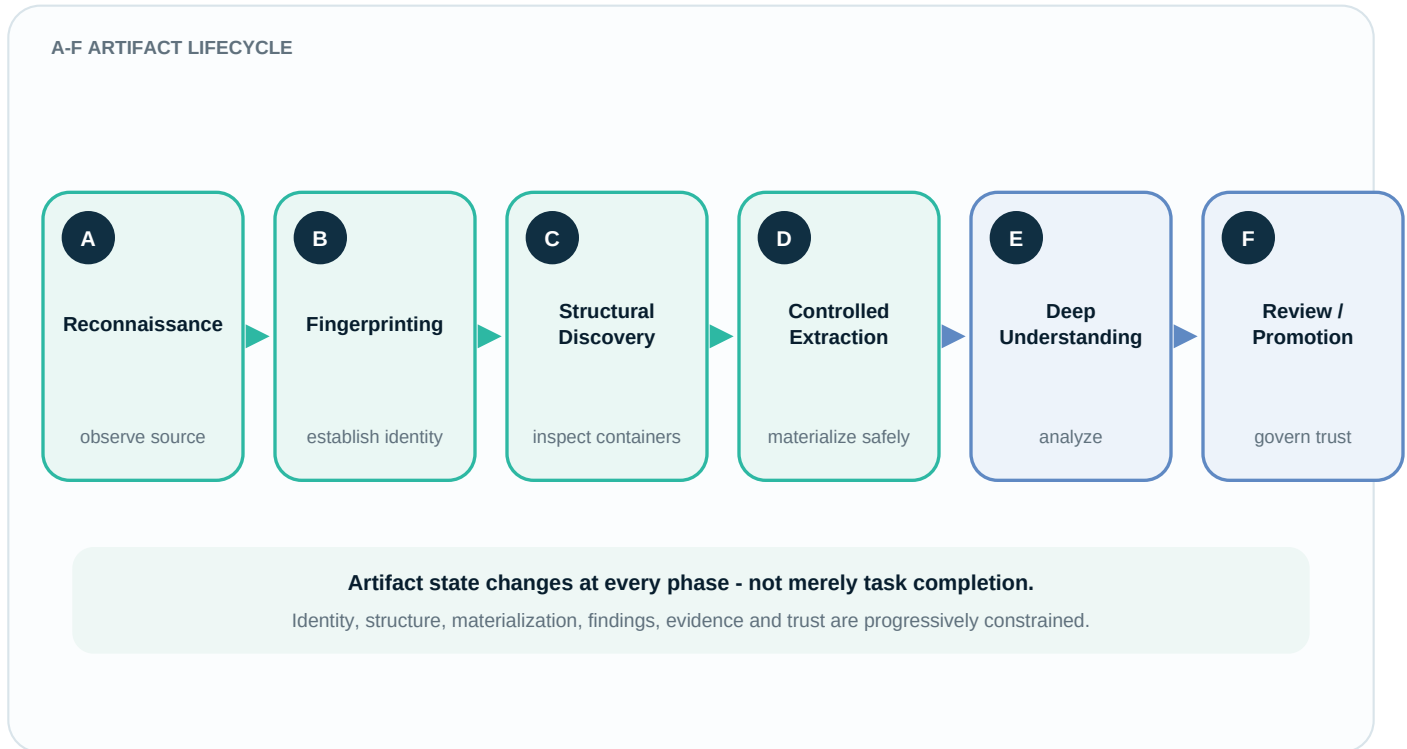
Can the conclusion be reproduced?

Recover the exact bytes, analyzer version, configuration, events, and lifecycle checkpoints.

Successful completion means ATLAS can preserve the chain from raw source and exact bytes, through every structural and analytical operation, to the evidence produced, the decision applied, and every trusted destination that received it.

Rendered Mermaid 01 - Lifecycle

The lifecycle view shows the stable semantic progression that governs how an artifact moves from initial observation to a defended trust decision.



WHAT THIS DIAGRAM REPRESENTS

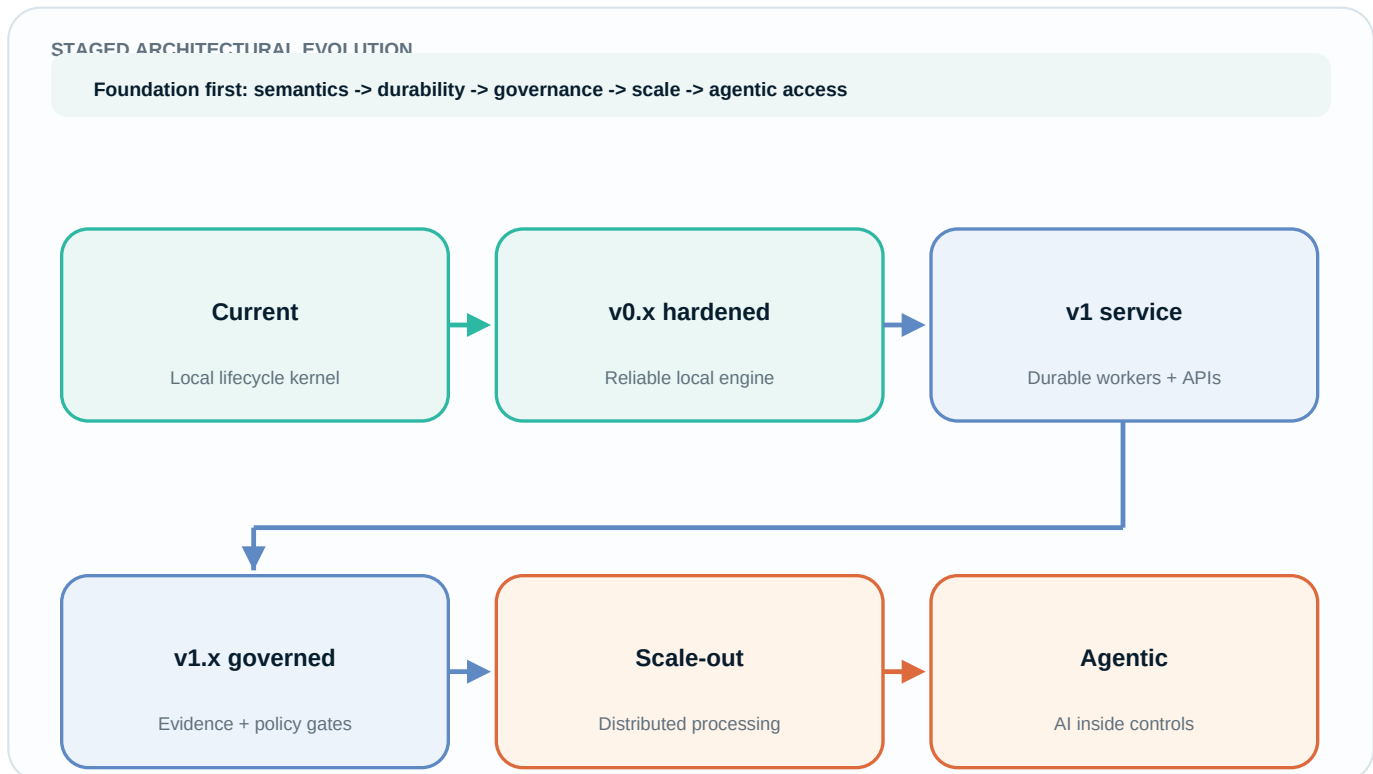
This diagram shows the stable six-stage semantic path an artifact follows from initial observation to a governed trust decision. Each transition represents a change in what ATLAS knows about the artifact or is authorized to do with it, rather than a generic dependency between tasks. Structural Discovery deliberately precedes Controlled Extraction so container risk can be evaluated before potentially hostile content is materialized on disk. Deep Understanding remains an extensible analysis boundary where tools, parsers, security engines, or models can produce normalized findings without inheriting authority over the lifecycle. Review / Promotion is the final trust gate, ensuring that analyzer output does not automatically become authoritative downstream data.

DESIGN IMPLICATION

Preserve this A-F sequence as the semantic spine even if future implementations add workers, queues, APIs, or internal DAG behavior.

Rendered Mermaid 02 - Evolution Roadmap

The evolution view shows how ATLAS should mature in controlled stages, adding durability and reach without changing the meaning of the lifecycle itself.



WHAT THIS DIAGRAM REPRESENTS

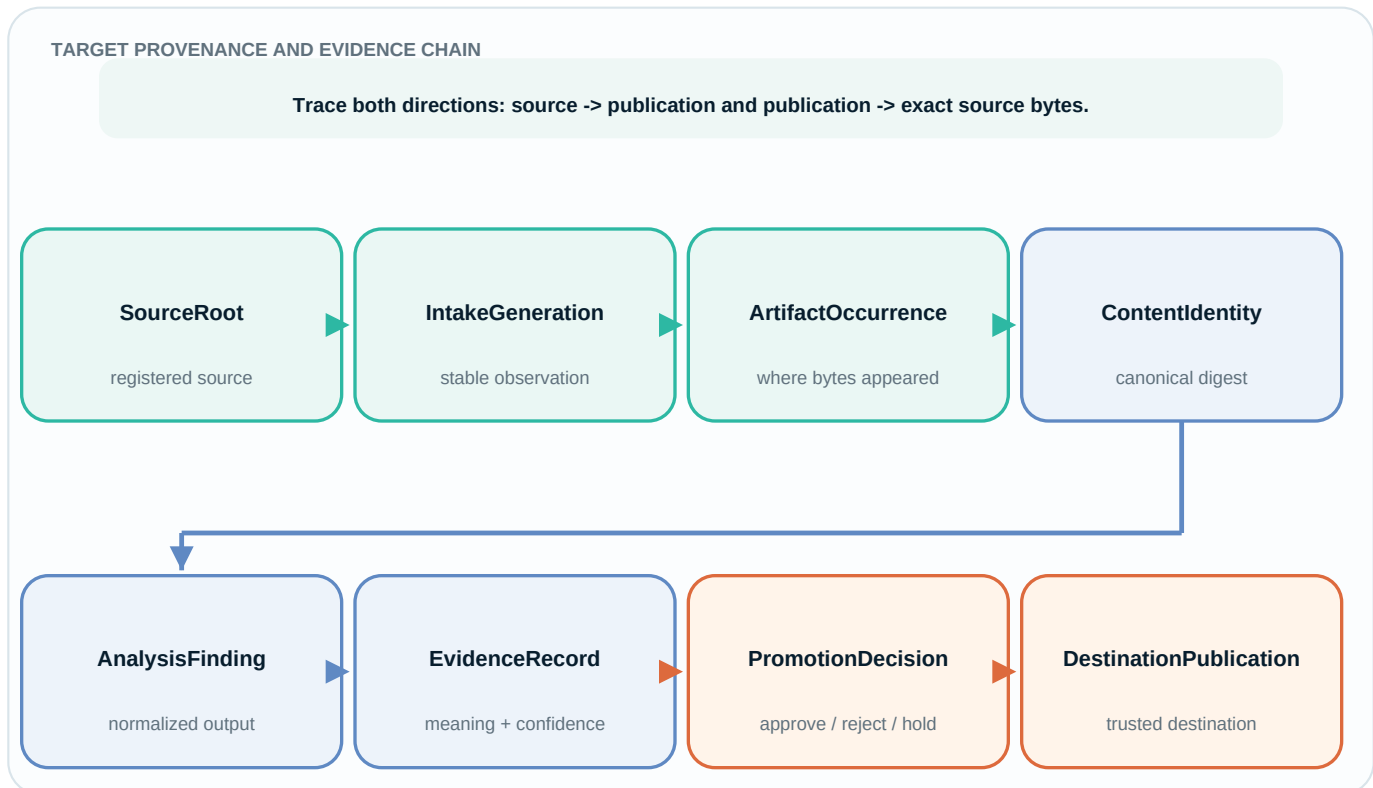
This diagram shows ATLAS maturing through staged capability, from the current local lifecycle kernel to a hardened local engine, then to a durable service, governed control plane, distributed scale-out, and finally agentic integration. Each stage adds infrastructure and operational reach without changing the A-F lifecycle semantics that define identity, safety, analysis, evidence, and trust. Semantic integrity, durable state, recovery behavior, and control requests come before service interfaces and distributed workers so unresolved local behavior is not multiplied across more components. Governance is introduced before broad scale because persistent evidence, review decisions, quarantine, and policy gates are what make larger deployments defensible. Agentic integration appears last because agents should select or parameterize governed workflows, not replace ATLAS as the authority over provenance, policy, lifecycle state, or final promotion.

DESIGN IMPLICATION

Treat scale as an infrastructure change, not a semantic redesign; every later stage should preserve the same provenance and trust contracts proven locally.

Rendered Mermaid 03 - Target Lineage Chain

The lineage view shows the target evidence chain that lets a trusted downstream publication be traced back to the exact source material that produced it.



WHAT THIS DIAGRAM REPRESENTS

This diagram shows the target traceability chain that allows ATLAS to follow information from a registered source through to a final downstream publication. SourceRoot and IntakeGeneration establish where and when material was observed, while ArtifactOccurrence and ContentIdentity deliberately separate location from the exact bytes that were processed. AnalysisFinding and EvidenceRecord capture what a tool, parser, security engine, or model concluded and how that conclusion is characterized, including meaning, provenance, and confidence where applicable. PromotionDecision records the governance step that determines whether the evidence or artifact is approved, rejected, or held for further review. DestinationPublication closes the chain by recording where approved material was sent, enabling full bidirectional lineage from a trusted output back to the original observed bytes.

DESIGN IMPLICATION

This model is the foundation for reproducibility, deduplication without provenance loss, auditability, evidence review, and trustworthy knowledge publication.

Diagram 04 - ATLAS Capability Mind Map

This mind map clusters ATLAS around its core system concerns so the product can be discussed as a coherent platform rather than a loose collection of utilities.



WHAT THIS DIAGRAM REPRESENTS

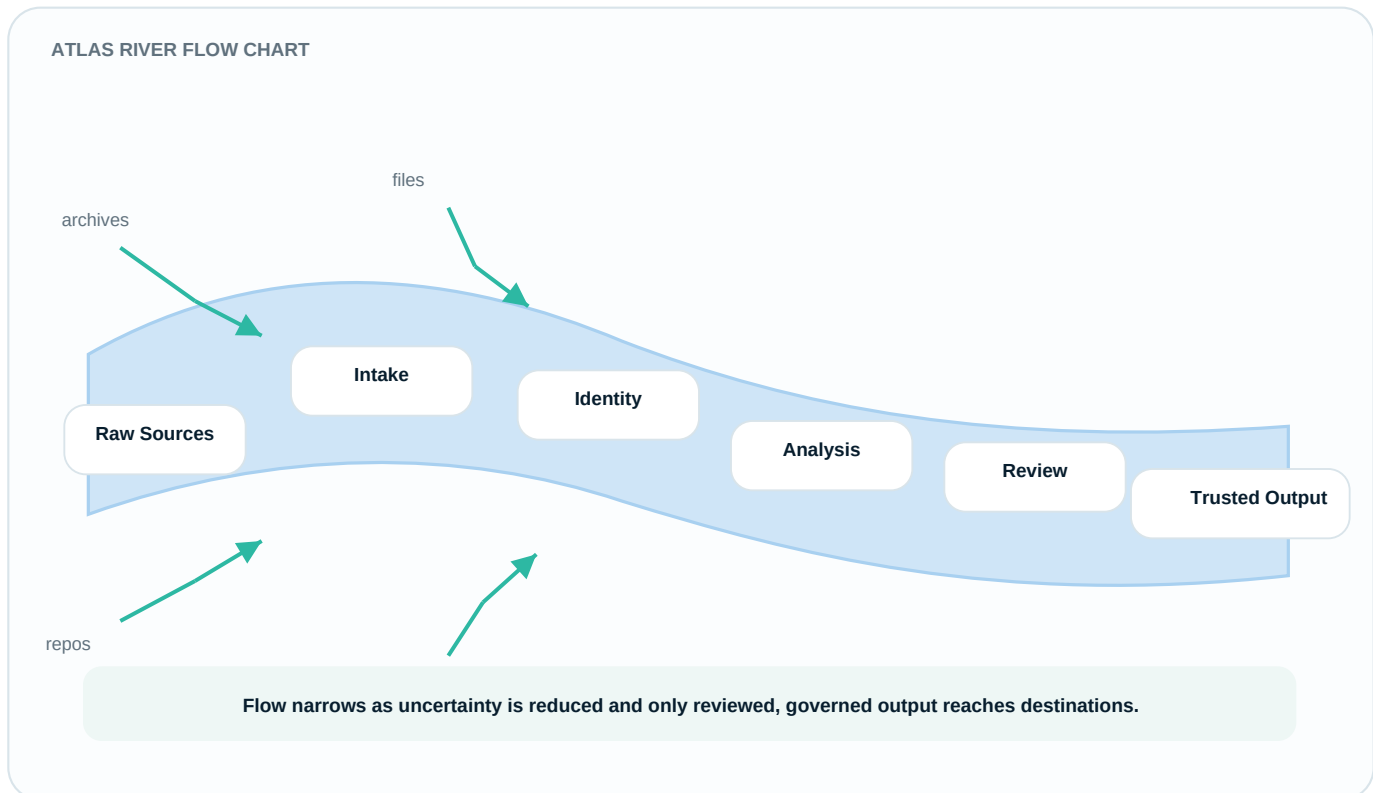
This mind map shows ATLAS as a central control-plane concept with six capability domains radiating outward: Identity, Safety, Lifecycle, Evidence, Integrations, and Use Cases. The purpose of the visual is to show that the platform is defined not by one tool or one workflow, but by the interaction between provenance, controlled execution, governance, and extensibility. Each branch represents a family of responsibilities that can grow over time while remaining anchored to the same lifecycle and trust model at the center. The map also helps stakeholders see where future features belong, which reduces architectural drift and prevents important capabilities from being treated as disconnected add-ons.

DESIGN IMPLICATION

Use this view when aligning stakeholders, scoping roadmaps, or explaining why ATLAS should be built as a governed platform rather than a point solution.

Diagram 05 - ATLAS River Flow Chart

The river-flow view expresses ATLAS as a continuously narrowing stream in which raw inputs are progressively filtered, enriched, reviewed, and routed into trusted outcomes.



WHAT THIS DIAGRAM REPRESENTS

This river flow chart shows ATLAS as a stream that begins with many raw inputs and becomes narrower as uncertainty is reduced through intake, identity, analysis, and review. The tributaries depict the fact that different source types can enter the system, but they are brought into a common controlled channel rather than handled as separate ad hoc pipelines. Each stage along the river represents progressive normalization and governance, meaning the system is always converting noisy observations into more structured and defensible information. By the time the flow reaches trusted output, the visual makes clear that downstream publication is not a direct consequence of ingestion, but the result of staged processing and explicit control.

DESIGN IMPLICATION

Use this view to explain ATLAS to non-specialists because it emphasizes progressive reduction of uncertainty and the disciplined movement from raw source material to trustworthy output.

Protect the part of ATLAS that is genuinely differentiated

Do not allow ATLAS to become another agent framework. Preserve deterministic artifact progression as the core abstraction.

ATLAS has a legitimate architectural reason to exist because it can become the reusable, observable, evidence-aware boundary between raw or untrusted artifacts and trusted downstream systems.

Protect these invariants

- The six-phase lifecycle remains explicit and ordered.
- Exact content identity is distinct from occurrence/location.
- Materialization is controlled and isolated.
- Analyzer output is not automatically trusted.
- Evidence and promotion decisions are durable and attributable.
- External APIs and agents cannot bypass lifecycle policy.

Use scale to amplify - not redefine

- Workers distribute execution, not authority.
- PostgreSQL scales state, not meaning.
- CAS/object storage scales content, not provenance semantics.
- MCP/REST widen access, not privilege.
- AI increases analytical capability, not trust authority.
- UI surfaces decisions; it does not become the source of truth.

DEVELOPMENT DIRECTION

Build ATLAS into a governed artifact-processing substrate. Finish semantic integrity and durable control first; operationalize evidence and analyzers second; scale infrastructure third. The payoff is one shared lifecycle that BinReaper, RUDI-K, repository analysis, forensics, malware/network analysis, corpus curation, supply-chain workflows, and future agents can reuse without independently rebuilding identity, safety, provenance, and trust controls.

Prepared from the supplied ATLAS development-direction source. Repository reference in source: Runndownn/ATLAS, main @ efa547cafc64b58d60d19dcc1e5ba32c6b046bc5.